МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«Северо-Кавказский федеральный университет»

Отчет по лабораторной работе № 2

«Ветвления и циклы в Python»

по дисциплине «Программирование на языке Python»

Выполнил студент группы ЭНЭ-б-о-25-1
Величко С.Р. « »_____20__г.
Подпись студента_____
Работа защищена « »_____20__г.
Проверила Хацукова А.И. _____
(подпись)

Ставрополь 2026

Цель работы: освоить выбор действий с помощью if-elif-else и простейшего match-case, перебор с помощью for и range(), циклы while, операторы break и continue; научиться проверять границы условий и завершение циклов.

ОС: Windows 11
Версия Python: 3.15.2
Ссылка на GitHub: https://github.com/N-cat-210/python-lab-02

Задание 1: (Вариант – 4) С клавиатуры вводится натуральное число n. В зависимости от значения остатка r при делении числа n на 7 вывести на экран число n в виде n = 7*k + r, если r = 0 вывести на экран n = 7*k.

```
E: > Python > individual_01.py > ...
#Вариант 4
n2 = (input("Введите натуральное число: "))
try:
    n1 = float(n2)
    if True:
        n = int(n1)
        if n/n1 != 1:
            print("Ошибка! Вы ввели не натуральное число!")
        else:
            r = n%7
            k=n//7
            if r ==0:
                print(f"{n}=7*{k}")
            else:
                print(f"{n}=7*{k}+{r}")
except ValueError:
    print("Ой, похоже это не число!")
```

Рисунок 1. Листинг программы individual_01.py

```
PS C:\Users\User\AppData\Local\Programs\Microsoft VS Code>
Введите натуральное число: Восемь
Ой, похоже это не число!
PS C:\Users\User\AppData\Local\Programs\Microsoft VS Code>
Введите натуральное число: 45
45=7*6+3
PS C:\Users\User\AppData\Local\Programs\Microsoft VS Code>
Введите натуральное число: 456.56
Ошибка! Вы ввели не натуральное число!
```

Рисунок 2. Результаты

Задание 2: (Вариант – 4) Напечатать три данных числа a, b и c сначала в порядке их возрастания, а затем – в порядке убывания. Использовать сравнения и перестановки.

```
a = int(input("a: "))
b = int(input("b: "))
c = int(input("c: "))
if a>b>c:
    print(f"{c},{b},{a}")
    print(f"{a},{b},{c}")
if a>c>b:
    print(f"{b},{c},{a}")
    print(f"{a},{c},{b}")
if b>a>c:
    print(f"{c},{a},{b}")
    print(f"{b},{a},{c}")
if b>c>a:
    print(f"{a},{c},{b}")
    print(f"{b},{c},{a}")
if c>a>b:
    print(f"{b},{a},{c}")
    print(f"{c},{a},{b}")
if c>b>a:
    print(f"{a},{b},{c}")
    print(f"{c},{b},{a}")
```

Рисунок 3. Листинг

```
a: 6
b: 7
c: 56
6,7,56
56,7,6
```

Рисунок 4. Результаты

Задание 3: (Вариант – 4) Через сколько дней спортсмен будет пробегать больше 15 км в день, если в первый день бежал 10 км, а затем на 10% больше, чем в предыдущий день?

```
p = 10
c = 1
while p < 15:
    p = p + (p/100)*10
    c = c + 1
print(c)
```

Рисунок 5. Листинг

Ответ – 6.

Задание 4: (Вариант – 4)

4. **Интегральный гиперболический косинус.** Допустимый диапазон: $0.1 \le x \le 1$.

$$\mathrm{Chi}(x) = \gamma + \ln x + \int_0^x \frac{\cosh t - 1}{t}\,dt = \gamma + \ln x + \sum_{k=1}^{\infty} \frac{x^{2k}}{(2k)(2k)!}.$$

Рисунок 6 - Задание

```
import math
import sys

if __name__ == "__main__":
    x = float(input("x (от 0.1 до 1): "))
    if not 0.1 <= x <= 1:
        print("Ошибка: требуется 0.1 <= x <= 1", file=sys.stderr)
        sys.exit(1)

    euler_gamma = 0.5772156649015329
    eps = 1e-10
    max_terms = 1000

    k = 1

    term = (x ** 2) / 4.0
    total = term

    while term > eps:
        if k >= max_terms:
            print("Превышен предел числа членов ряда", file=sys.stderr)
            sys.exit(1)

        term *= (x ** 2) * (2 * k) / ((2 * k + 1) * (2 * k + 2) ** 2)
        k += 1
        total += term

    result = euler_gamma + math.log(x) + total

    print(f"Chi({x}) = {result:.10f}")
    print(f"Членов ряда: {k}")
    print(f"Оценка остатка ряда: {term:.3e}")
```

Рисунок 7 - Код

```
x (от 0.1 до 1): 0.56987
Chi(0.56987) = 0.0971631482
Членов ряда: 5
Оценка остатка ряда: 9.954e-11
```

Рисунок 7 - результат

Вывод: освоил выбор действий с помощью if-elif-else и простейшего match-case, перебор с помощью for и range(), циклы while, операторы break и continue; научиться проверять границы условий и завершение циклов.

Ответы на контрольные вопросы:

1. Присваивание (=) записывает значение в переменную. Сравнение (==) проверяет равенство двух значений и возвращает True или False.
2. Ложными считаются False, None, пустые коллекции ([], {}, ""). Строка "0" истинна, так как это непустая строка (содержит символ).
3. and (И) — истинно, если оба истинны. or (ИЛИ) — истинно, если хотя бы одно истинно. not (НЕ) — инвертирует. Сокращенное вычисление: интерпретатор прекращает проверку, как только результат ясен (например, в and при первом False).
4. Несколько if проверяются все независимо друг от друга. Цепочка if-elif-else выполнит только один подходящий блок (первый сверху вниз), остальные будут пропущены.
5. При пересечении диапазонов программа выполнит инструкцию для первого подходящего условия. Если более широкий диапазон стоит первым, он перехватит значения, и до узкого диапазона очередь не дойдет.
6. Это выражение, которое вычисляется в True или False. Удобно для присваивания результата проверки переменной или для возврата из функции.
7. С версии Python 3.10 (PEP 634).
8. Числовой 1 — это тип int (целое число). Строковый "1" — это тип str (текст), с ним нельзя проводить математические операции без конвертации.
9. case _ — это шаблон по умолчанию (аналог else). Он размещается последним, так как match выполняет первое совпадение сверху вниз, а _ совпадает с чем угодно.
10. Через вертикальную черту (логическое ИЛИ): case 1 | 2 | 3:.
11. Не нужен. В Python match автоматически выходит после выполнения первого совпавшего блока (в отличие от switch в C/Java).
12. Потому что case ищет по шаблону (значению), а переменная — это имя, куда можно записать значение (захват). Сравнение с переменной требует указания точки (например, case Константы.NAME:).

13. if — для сложной логики, диапазонов и вычислений. match — для проверки на соответствие конкретным шаблонам, структурам данных или перечислениям.

14. Функция возвращает объект range. Это не список, а ленивый итерируемый объект, который генерирует числа по мере необходимости, экономя память.

15. range(15, 0, -2). Ноль отсутствует, так как правая граница в Python не включается, а шаг равен -2 (после 1 идет -1, но цикл останавливается).

16. Если условие изначально ложно (например, while False: или если счетчик уже превышает лимит).

17. break полностью прерывает цикл. continue пропускает оставшуюся часть текущей итерации и переходит к следующей. Оба действуют только на ближайший вложенный цикл.

18. Да, если счетчик, который меняется в цикле, находится после инструкции continue, он не будет обновляться.

19. Блок else в цикле выполняется только в том случае, если цикл завершился сам (условие стало ложным), а не был прерван через break.

20. Сумма: 0. Произведение: 1. (Это нейтральные элементы для сложения и умножения).

21. Условие останова проверяет, когда прекратить вычисления (например, abs(x) < eps). Оценка погрешности — это теоретическое вычисление того, насколько текущий результат отличается от точного ответа.

22. Из-за особенностей хранения чисел с плавающей точкой в памяти прямое сравнение a == b может дать False даже для очень близких чисел. math.isclose использует допуск (погрешность).

23. Использовать тесты на граничные значения (например, для 0.1 <= x <= 1 проверить 0.0999, 0.1, 1.0, 1.0001) и внутри диапазона.

24. Трассировочная таблица показывает пошаговое изменение значений переменных по ходу программы. Она дополняет конечный вывод, показывая, *как* программа пришла к результату и где могла возникнуть ошибка.

25. stdout — стандартный поток вывода (обычные сообщения). stderr — поток вывода ошибок. Код завершения — статус программы (0 — успех, не 0 — ошибка). Сообщение об ошибке (через print или sys.stderr) не останавливает программу автоматически, для остановки нужен sys.exit() или вызов исключения (raise).